

SOFTWARE CONSTRUCTION AND DEVELOPMENT

Engr. Muhammad Umer Haroon

SOFTWARE CONSTRUCTION AND DEVELOPMENT

WEEK

RECALL LAST WEEKS

Function Name Matters

- Ward's principle: "You know you are working on clean code when each routine turns out to be pretty much what you expected." Half the battle to achieving that principle is choosing good names for small functions that do one thing.
- The smaller and more focused a function is, the easier it is to choose a descriptive name.

GIVE ME EXAMPLES ?

- A long descriptive name is better than a short enigmatic name.
- A long descriptive name is better than a long descriptive comment. Use a naming convention that allows multiple words to be easily read in the function names, and then make use of those multiple words to give the function a name that says what it does.

Argument vs parameter ?

Function Arguments

- The ideal number of arguments for a function is zero (niladic).
- Next comes one (monadic), followed closely by two (dyadic).
- Three arguments (triadic) should be avoided where possible. More than three (polyadic) requires very special justification and then shouldn’t be used anyway.

- Arguments are even harder from a testing point of view. Imagine the difficulty of writing all the test cases to ensure that all the various combinations of arguments work properly.
- With two arguments the problem gets a bit more challenging. With more than two arguments, testing every combination of appropriate values can be daunting

```
def calculate_area(shape, length, width=None, radius=None):

    if shape == 'rectangle':
        if width is None:
            raise ValueError("Width is required for rectangles.")
        return length * width
    elif shape == 'circle':
        if radius is None:
            raise ValueError("Radius is required for circles.")
        return 3.14159 * radius ** 2
    else:
        raise ValueError("Invalid shape. Choose 'rectangle' or 'circle'.")

print(calculate_area('rectangle',1 , 0, 5))
```

Argument objects

When a function seems to need more than two or three arguments, it is likely that some of those arguments ought to be wrapped into a class of their own. Consider, for example, the difference between the two following declarations:

```
Circle makeCircle(double x, double y, double radius);
Circle makeCircle(Point center, double radius);
```

Reducing the number of arguments by creating objects out of them may seem like cheating, but it's not. When groups of variables are passed together, the way x and y are in the example above, they are likely part of a concept that deserves a name of its own.

One Entry and One Exit

- Some programmers follow Edsger Dijkstra's rules of structured programming. Dijkstra said that every function, and every block within a function, should have one entry and one exit.
- Following these rules means that there should only be one return statement in a function, no break or continue statements in a loop, and never, ever, any goto statements

Nested Functions

- Many programming languages support nested functions, which are functions defined within other functions.
- ALGOL ,Pascal ,Modula 2 ,Oberon ,Ada, Python, Swift, D,JavaScript, PHP
- C, C++ ,C# do not directly support nested functions

Nested Functions in Python

- Inner functions, also known as nested functions, are functions that you define inside other functions.

```
def outer_func():
    def inner_func():
        print("Hello, World!")
    inner_func()


outer_func()
```

```
def outer_func(who):
    def inner_func():
        print(f"Hello, {who}")
    inner_func()


outer_func("World!")
```

Regarding code from last slide

- We pass a string as an argument to outer_func(), and inner_func() will access that argument through the name who. This name, however, is defined in the local scope of outer_func().
- The names that you define in the local scope of an outer function are known as **nonlocal names**. They are nonlocal from the inner_func() point of view.

Avoid Recursive Functions

- Aim to minimize the number of recursive calls within a function. Too many recursive calls can make the code harder to understand and debug.
- If possible, try to solve the problem iteratively instead of recursively. Iteration is often more efficient and easier to reason about.
- Limit the depth of recursive calls to avoid excessive nesting. Deep recursion can make the code harder to read and understand.

Debatable :

https://www.reddit.com/r/learnprogramming/comments/12e6sk/best_practices_recursion_why_is_it_generally/

5 Brief Explanation About The Difference Between Recursion Vs Iteration – Slash

functional programming - When will iteration be more readable than recursion? - Stack Overflow

Tail Recursion

- Tail recursion is a specific form of recursion where the recursive call is the last operation performed in the function. In other words, if a function calls itself as its last action, it is considered a tail recursive function.

- Tail recursion is important because many programming languages can optimize tail recursive calls to avoid growing the call stack. This optimization is known as Tail Call Optimization (TCO). When TCO is applied, the current function's stack frame can be reused for the next function call, preventing stack overflow errors and improving performance.

Non-Tail Recursive Function

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(4))
```

Let us trace factorial(4). When you call factorial(4), it cannot return yet because it needs the result of factorial(3) first, and then it must multiply that result by 4. So it stays on the stack. Then factorial(3) stays because it needs factorial(2), and so on. The stack looks like this while waiting.

Tail Recursive Function

```
def tail_recursive_factorial(n, accumulator=1):
    if n == 0:
        return accumulator
    else:
        return tail_recursive_factorial(n - 1, n * accumulator)
print(tail_recursive_factorial(4))
```

The call itself is the return value. The multiplication happens before the call, inside the arguments. Once factorial_tail(n - 1, acc * n) is called, the current frame has nothing left to do. It could theoretically be thrown away.

Does python supports TCO ?
READ YOUR SELF

- https://stackoverflow.com/questions/310974/what-is-tail-call-optimization
- https://stackoverflow.com/questions/13591970/does-python-optimize-tail-recursion
- https://stackoverflow.com/questions/890461/whats-the-big-deal-with-tail-call-optimization-and-why-does-python-need-it

interesting insights

- Tail Recursion: Python does not optimize tail recursion, meaning that even if a recursive function is designed to be tail-recursive, it will still hit the recursion limit if the depth is too great.
- That is 1000 ….

- In fact, TCO essentially turns tail recursion into iteration behind the scenes.
- You can change the recursion limit with sys.setrecursionlimit, but this is dangerous because it can cause a real stack overflow and crash the interpreter. It is not a substitute for TCO.

- **Reusability is not an accident , it's a design decision made at the moment you write each function.**

Call Stack

- When a function calls another function, Python needs to remember where to come back to once the inner function finishes.
- It stores this information in a structure called the call stack. Each time a function is called, a new frame is pushed onto the stack.
- When the function returns, its frame is popped off. If a function calls itself many times without returning, the stack keeps growing, and eventually Python raises a RecursionError.

Normal Recursion

- In normal recursion, the recursive call is not the last thing the function does. After the recursive call returns, there is still work left to do, such as multiplying, adding, or combining results.
- Because of this leftover work, the current frame cannot be thrown away. It has to stay on the stack until the inner call finishes, so that it can complete its own work afterward.

For queries

.....